

NYU Fall Semester 2023

Lecture 13: Concurrency II

Professor G. Sandoval

13

From Last Class

- Race Conditions
- Critical Sections
- How to avoid Races:
 - Spinlocks
 - Mutex
- How to implement:
 - Spinlocks
 - Mutexes

Sleep and Wakeup

- Instead of waiting in a loop, wasting CPU, we would like to put the waiting process to sleep, waking up when the lock is released
- Often, sleep and wakeup take as parameters the address of some variable so we can match up sleeps with wakeups
- E.g., in xv6:
 - `sleep(void *chan)`
 - `wakeup(void *chan)`

Producer-Consumer

- Imagine we have two tasks, one that *produces* items and places them in a fixed-size buffer, and one that *consumes* them
 - An example you have seen already – a pipe!
- If the buffer is full, the producer sleeps until there's space
- If the buffer is empty, the consumer sleeps until there's data available

Mutually Exclusive Access



Producer



Shared Resource



Consumer

The Producer-Consumer Problem

```
#define N 100                                /* number of slots in the buffer */
int count = 0;                               /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();              /* repeat forever */
        if (count == N) sleep();             /* generate next item */
        insert_item(item);                   /* if buffer is full, go to sleep */
        count = count + 1;                   /* put item in buffer */
        if (count == 1) wakeup(consumer);    /* increment count of items in buffer */
        /* was buffer empty? */
    }
}
```

```
void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();              /* repeat forever */
        item = remove_item();                 /* if buffer is empty, got to sleep */
        count = count - 1;                    /* take item out of buffer */
        if (count == N - 1) wakeup(producer); /* decrement count of items in buffer */
        consume_item(item);                   /* was buffer full? */
        /* print item */
    }
}
```

The Lost Wakeup Problem

- Suppose the buffer is empty, and consumer checks that count is 0
- Just before consumer actually goes to sleep, it gets preempted, and producer puts something in the buffer
- Since count is now 1, producer tries to wake up consumer, but consumer isn't asleep yet, so it does nothing
- Control returns to the consumer, who goes to sleep, and never wakes up – it has missed its wakeup

Solving the Lost Wakeup Problem

- We can have the producer and the consumer share a lock
- The consumer acquires the lock, checks the value of count, and goes to sleep, passing the lock to the sleep function, which releases it
- The producer acquires the lock before calling wakeup; if the process is not yet fully asleep, it will wait, ensuring that the wakeup is sent *after* the process is actually asleep

Semaphores

- We saw last time that one way to solve the lost wakeup problem is by using a lock
- Another way to solve this problem is to use a semaphore: an integer to count the number of wakeups pending



Semaphore Operations

- Semaphores have two operations:
 - **down**: checks if semaphore is greater than 0, and decrements it if so; otherwise sleep
 - **up**: increments the value of the semaphore, and if it was previously zero, wakes up one of the sleeping processes at random
- Note that these operations are *atomic* and generally implemented as system calls
- Up never blocks; it always returns immediately

```

#define N 100                /* number of slots in the buffer */
typedef int semaphore,      /* semaphores are a special kind of int */
semaphore mutex = 1;        /* controls access to critical region */
semaphore empty = N;        /* counts empty buffer slots */
semaphore full 0;           /* counts full buffer slots */

void producer(void)
{
    int item;
    while (TRUE) {           /* TRUE is the constant 1 */
        item = produce_item(); /* generate something to put in buffer */
        down(empty);          /* decrement empty count */
        down(&mutex);          /* enter critical region */
        insert_item(item);     /* put new item in buffer */
        up(&mutex);            /* leave critical region */
        up(&full);             /* increment count of full slots */
    }
}

void consumer(void)
{
    int item;
    while (TRUE) {           /* infinite loop */
        down(&full);           /* decrement full count */
        down(&mutex);          /* enter critical region */
        item=remove_item();    /* take item from buffer */
        up(&mutex);            /* leave critical region */
        up(&empty);            /* increment count of empty slots */
        consume_item(item);    /* do something with the item */
    }
}

```

Barriers

- Sometimes we have algorithms that are a mix of parallel and sequential steps
- When going from the parallel to the sequential phase, we may need to stop, collate results, etc.
- We want a way to force a group of processes / threads to wait until all have finished

Barrier Example

- Suppose we have an algorithm that relies on repeatedly updating a matrix by multiplying it:

$$M_n = M_{n-1} * C$$

- The matrix multiplication can be parallelized by splitting it into submatrices
- But we can't update the final matrix until all subthreads have finished – so we need a barrier after each iteration

Parallel Matrix Multiplication

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$$

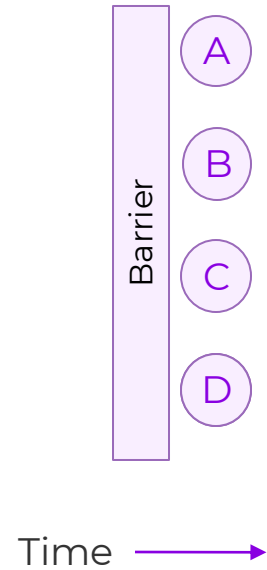
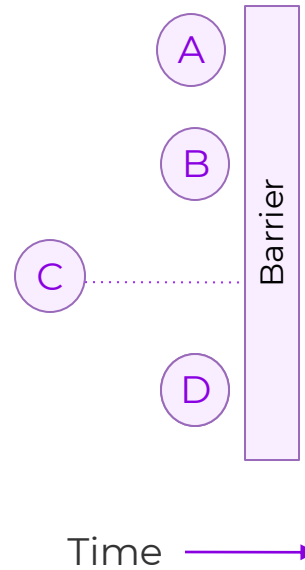
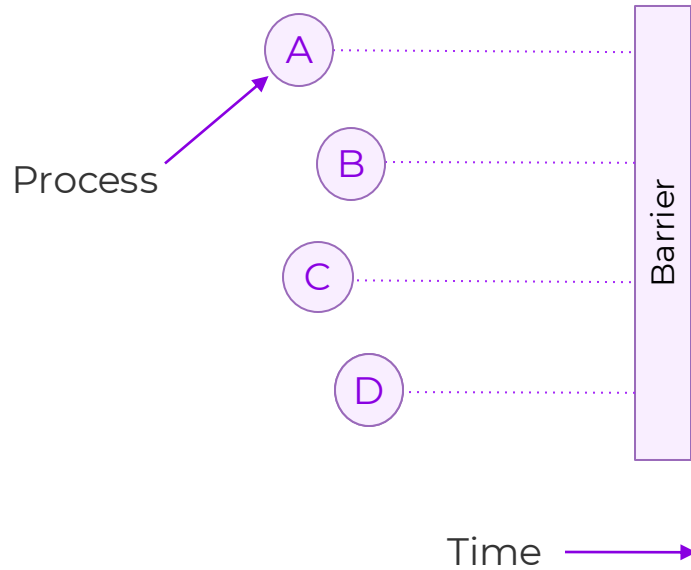
$$B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$$

$$A = \begin{bmatrix} a = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} & b = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \\ c = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} & d = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$$

$$B = \begin{bmatrix} e = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} & f = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \\ g = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} & h = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \end{bmatrix}$$

$$A * B = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

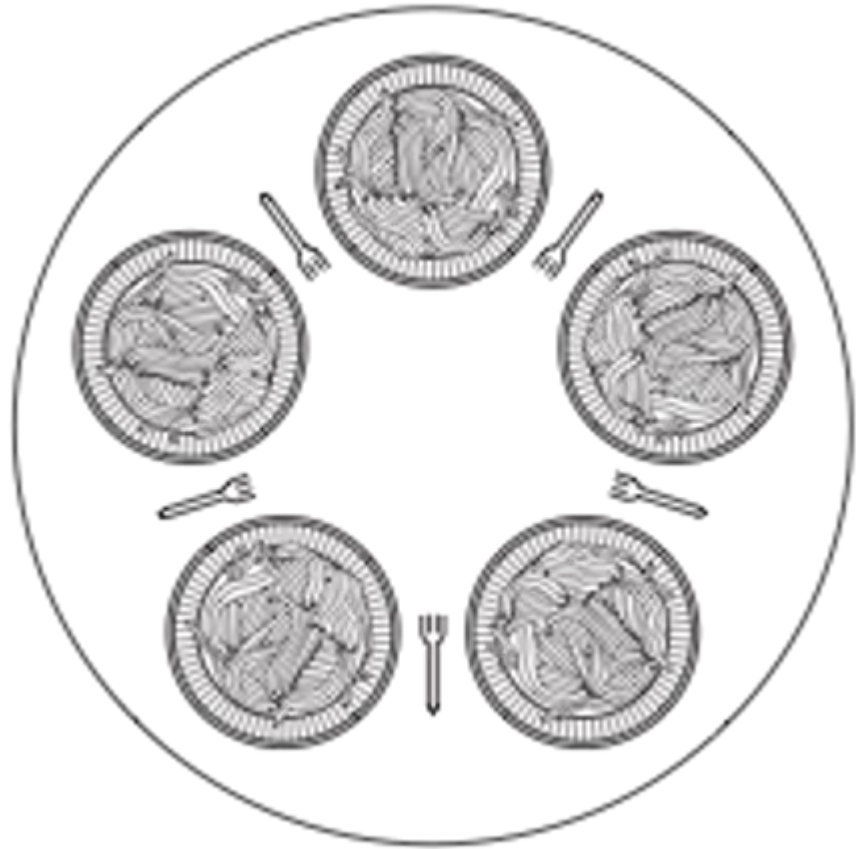
Barrier



Dining Philosophers Problem

- Classic synchronization problem that can be solved using many different synchronization primitives
- N philosophers around a table with N forks and N plates are either thinking or eating
- Eating requires two forks, the left fork and the right fork

Delicious Spaghetti



Dining Philosophers Problem

- A naïve solution: when a philosopher gets hungry, just pick up the left fork, then the right fork, waiting until each is available
- But: what happens if everyone picks up their left fork at once?

Naïve solution

```
#define N 5                                /* number of philosophers */

void philosopher(int i)                    /*i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think();                          /* philosopher is thinking */
        take_fork(i);                     /* take left fork */
        take_fork((i+1) % N);              /* take right fork; % is modulo operator */
        eat();                             /* yum-yum, spaghetti */
        put_fork(i);                       /* put left fork back on the table */
        put_fork((i+1) % N);               /* put right fork back on the table */
    }
}
```

Dining Philosophers Problem

Some solutions:

- Have a single mutex. When a philosopher wants to eat, acquire the mutex, then pick up the left and right forks, eat, and release the mutex.
 - But now only one philosopher can eat at a time, which is very inefficient
- If you can't get your fork, wait a random amount of time before trying again
 - But sometimes we need a deterministic algorithm that will never fail, no matter how many unlikely events happen

```

#define N          5          /* number of philosophers */
#define LEFT      (i+N-1)%N   /* number of i's left neighbor */
#define RIGHT     (i+1)%N     /* number of i's right neighbor */
#define THINKING  0          /* philosopher is thinking */
#define HUNGRY    1          /* philosopher is trying to get forks */
#define EATING    2          /* philosopher is eating */

typedef int semaphore;        /* semaphores are a special kind of int */
int state[N];                /* array to keep track of everyone's state */
semaphore mutex = 1;         /* mutual exclusion for critical regions */
semaphore s[N];              /* one semaphore per philosopher */

void philosopher(int i)      /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {           /* repeat forever */
        think();             /* philosopher is thinking */
        take_forks(i);       /* acquire two forks or block */
        eat();               /* yum-yum, spaghetti */
        put_forks(i);        /* put both forks back on table */
    }
}

```

```

void take_forks (int i)           /*i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                 /* enter critical region */
    state[i]= HUNGRY;             /* record fact that philosopher i is hungry */
    test(i);                     /* try to acquire 2 forks */
    up(&mutex);                   /* exit critical region */
    down(&s[i]);                  /* block if forks were not acquired */
}

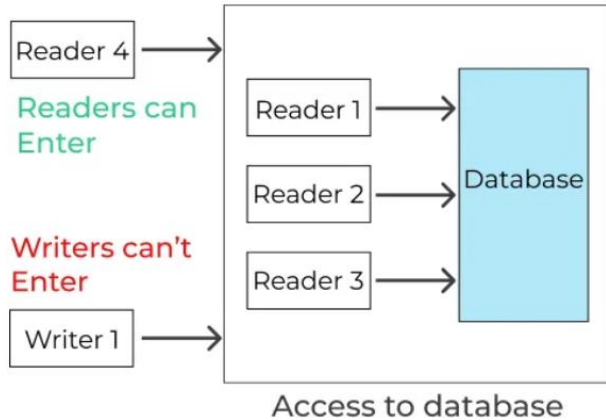
void put_forks(i)                 /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                 /* enter critical region */
    state[i]= THINKING;          /* philosopher has finished eating */
    test(LEFT);                  /* see if left neighbor can now eat */
    test(RIGHT);                 /* see if right neighbor can now eat */
    up(&mutex);                   /* exit critical region */
}

void test(i)                      /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i]= EATING;
        up(&s[i]);
    }
}

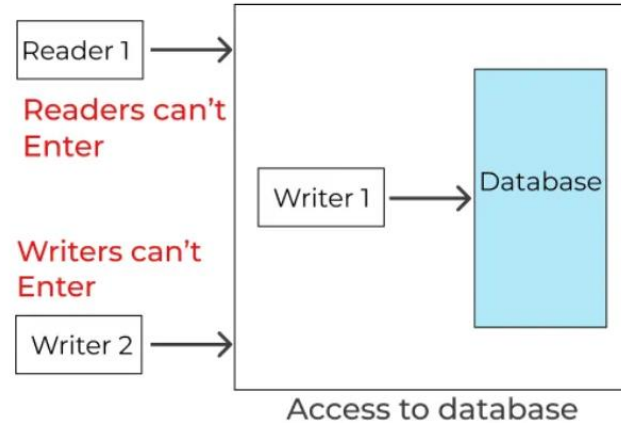
```

Readers and Writers Problem

When Readers are accessing the Database



When Writers are accessing the Database



Readers and Writers Problem

- Models shared access to a database, like an airline reservation system
- Multiple readers and writers
- It's okay for **more than one reader** to access the database at a time, but when a **writer** is writing **no one else** may access the database

Readers and Writers Problem

- To solve, we use two mutexes
 - One protects the count of the number of readers
 - One controls access to the database as a whole
- The first reader acquires the database mutex and increments the reader count
- Subsequent readers just increment the reader count
- The last reader to finish also releases the database lock
- Writers have to acquire database lock before writing

Readers and Writers Problem

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;

void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 0) up(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc - 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}

void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

```
/* use your imagination */
/* controls access to rc */
/* controls access to the database */
/* # of processes reading or wanting to */

/* repeat forever */
/* get exclusive access to rc */
/* one reader more now */
/* if this is the first reader ... */
/* release exclusive access to rc */
/* access the data */
/* get exclusive access to rc */
/* one reader fewer now */
/* if this is the last reader ... */
/* release exclusive access to rc */
/* noncritical region */

/* repeat forever */
/* noncritical region */
/* get exclusive access */
/* update the data */
/* release exclusive access */
```

Synchroni- zation in POSIX

- The standard way to do multithreading in C on POSIX (UNIX-like) systems is the *pthread* interface
- Allows creation & management of threads, and offers many standard synchronization primitives

pthread Basics

- Each thread is identified by a pthread_t data type

- Threads can be created using

```
int pthread_create(  
    pthread_t *thread,  
    const pthread_attr_t *attr,  
    void *(*start_routine) (void *),  
    void *arg  
);
```

- You can exit a thread by either returning from its start routine or calling
`void pthread_exit(void *retval);`

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NUM_THREADS 5
void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello World! It's me, thread #%ld!\n", tid);
    pthread_exit(NULL);
}
int main(int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for (t=0; t<NUM_THREADS; t++) {
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc) {
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    /* Last thing that main() should do */
    pthread_exit(NULL);
}

```

Coordinating Threads

- The main thread can *join* a thread to wait until it's finished

```
int pthread_join(pthread_t  
thread, void **retval);
```

- By doing this in a loop for all threads we can implement a *barrier*

Mutexes

- You can create a mutex with

```
int pthread_mutex_init(pthread_mutex_t *mutex, const  
pthread_mutexattr_t *attr);
```

- Statically allocated mutexes can just do:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

- Then, you have some standard operations on it:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Deadlocks

- Deadlocks are situations in which no work can get done because everyone is waiting to acquire a resource, e.g.:
 - Processes A and B, resources C and D
 - Process A locks C, process B locks D
 - Now A tries to lock D, and B tries to lock C
 - Deadlock – no one can make progress

Deadlocks: Resources

- To be general, we'll just think of abstract *resources* and processes that want to acquire them
- We have two flavors:
 - Non-preemptible: once given, they cannot be taken away without cooperation from the process
 - Preemptible: we can take it away from the process with no harm done
- For example, RAM is preemptible, because we can just swap the pages to disk without the process noticing

Deadlock Definition

- We can define a deadlock more formally now:

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

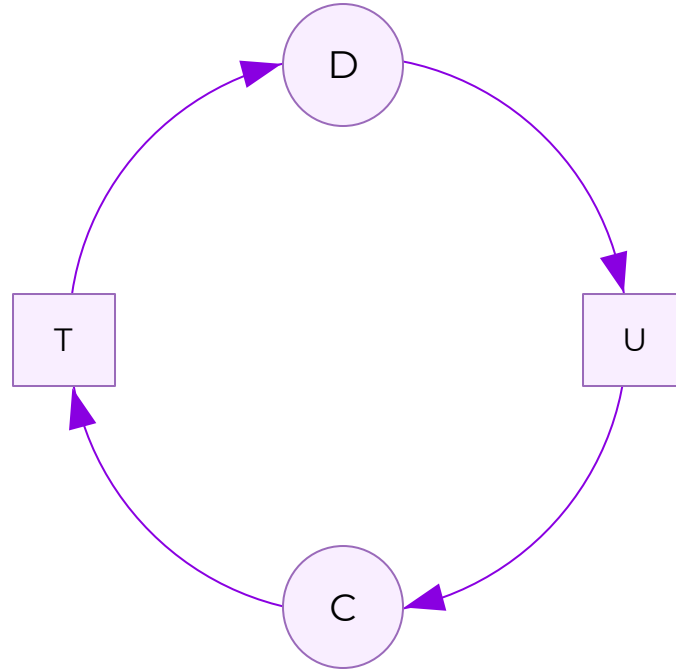
Deadlock Conditions

1. **Mutual exclusion condition** Each resource is either currently assigned to exactly one process or is available.
2. **Hold-and-wait condition** Processes currently holding resources that were granted earlier can request new resources.
3. **No-preemption condition** Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.
4. **Circular wait condition** There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member of the chain.

Modeling Deadlocks

- We can model deadlock conditions as a directed graph
- Two types of nodes: processes and resources
- An edge from process $\rightarrow$ resource indicates the process is blocked trying to acquire that resource
- An edge from resource $\rightarrow$ process indicates that the resource is currently held by the process

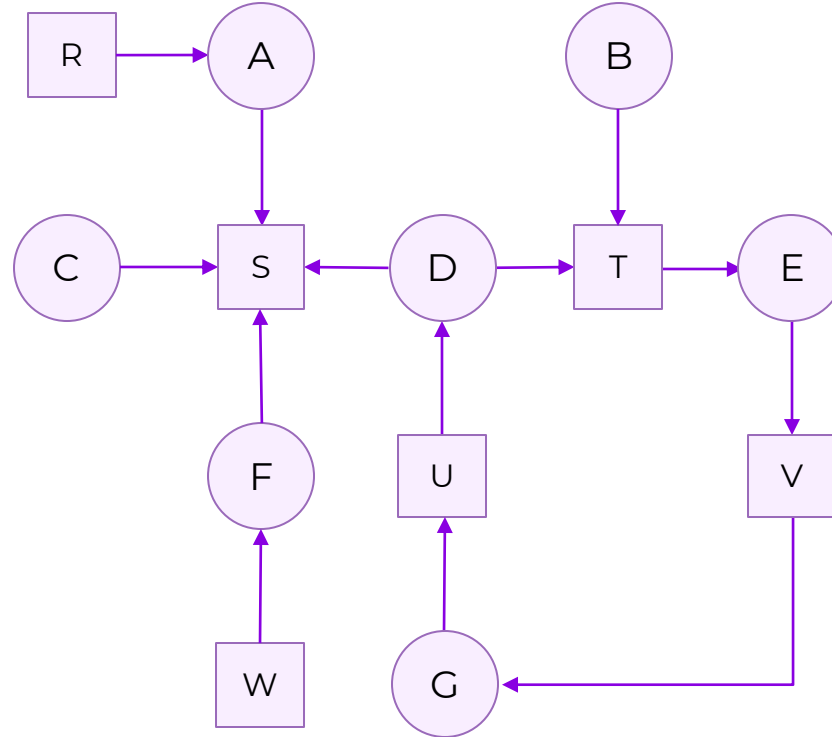
Deadlock Graph



 Resource

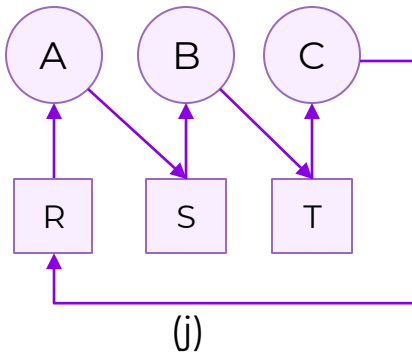
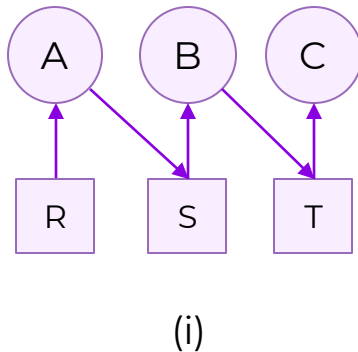
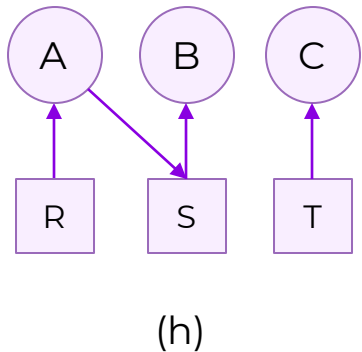
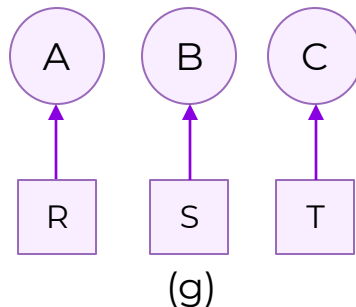
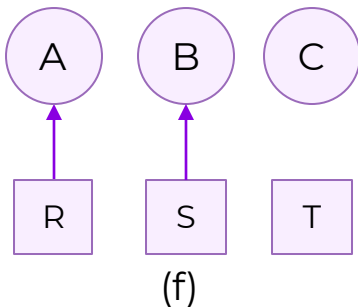
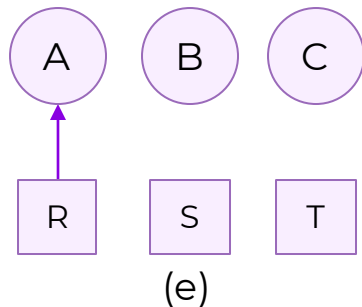
 Process

More Complicated

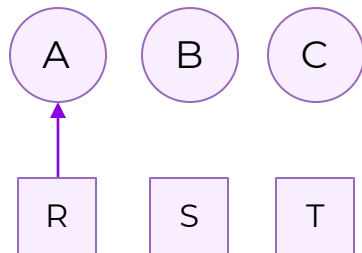


Deadlock Detection

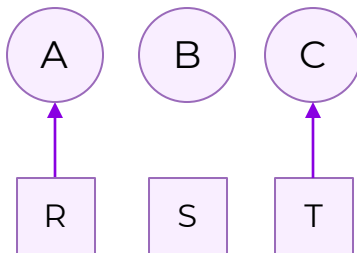
- We can see that having a deadlock is equivalent to having a cycle in our graph



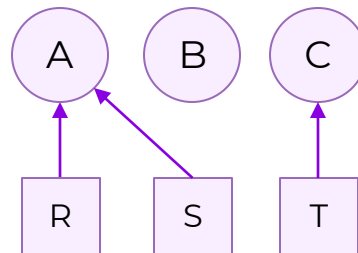
Deadlock Avoided



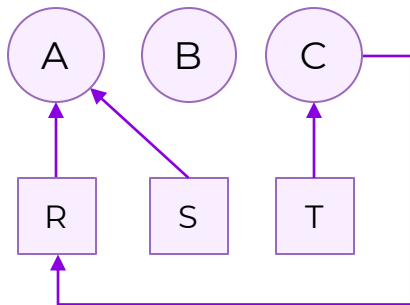
(l)



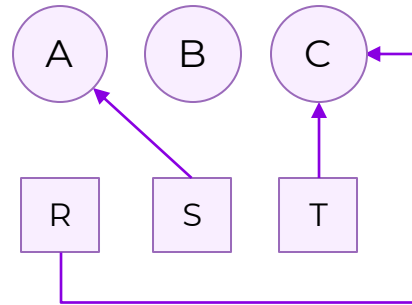
(m)



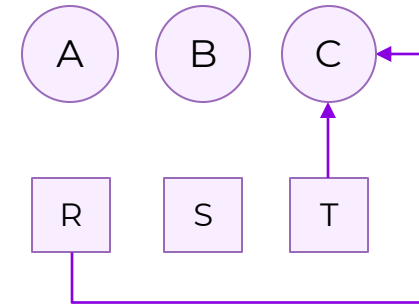
(n)



(o)



(p)



(q)

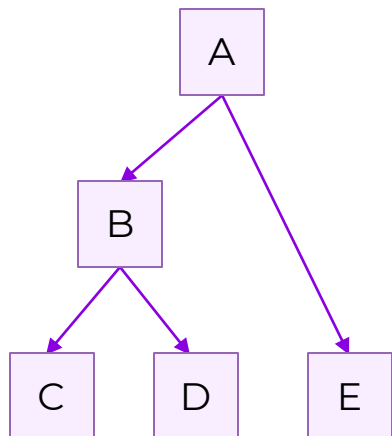
Dealing with Deadlocks

- Many strategies for dealing with deadlocks in a system:
 - Do nothing, just make sure you're careful and hope you don't make any mistakes that lead to a deadlock
 - Detect and recover
 - Avoid deadlocks by being careful about how we hand out resources
 - Prevent deadlocks by negating one of the four conditions we saw earlier

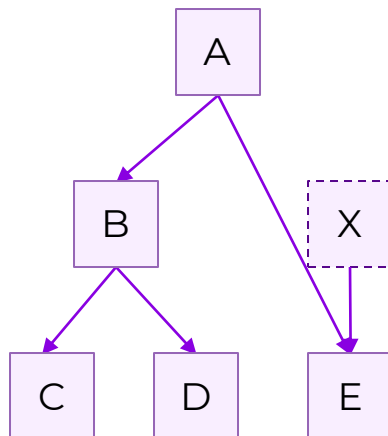
Read-Copy-Update

- If we're okay with some readers seeing an old version of a data structure for a while, we can sometimes avoid locking
- The trick is to update the structure but not free deleted items immediately – instead, wait until all readers are done

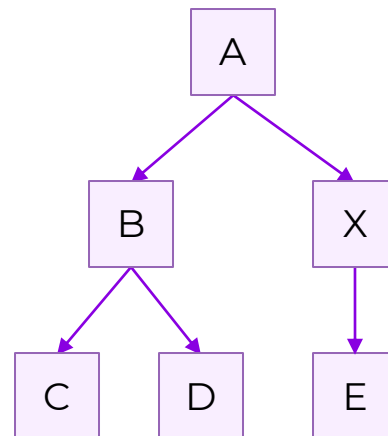
Adding a node:



(a) Original tree

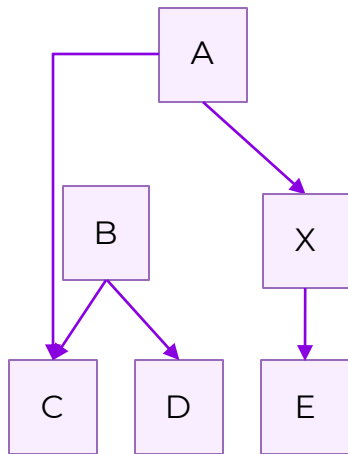


(b) Initialize node X and connect E to X. Any readers in A and E are not affected

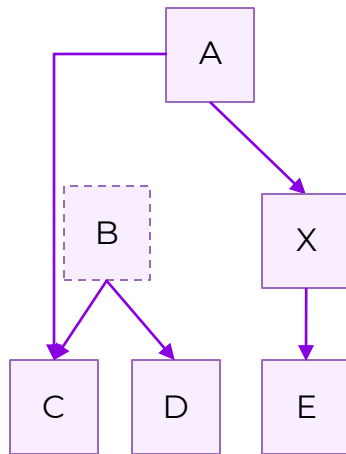


(c) When X is completely initialized, connect X to A. Readers currently in E will have read the old version, while readers in A will pick up the new version of tree

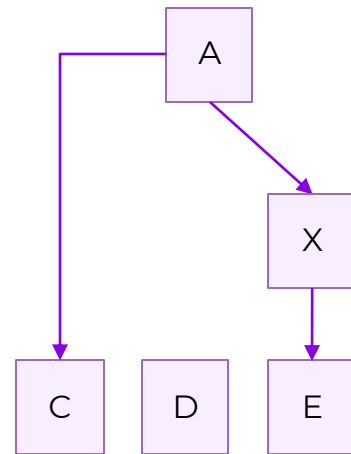
Removing nodes:



(c) Decouple B from A. Note that there may still be readers in B. All readers in B will see the old version of the tree, while all readers currently in A will see the new version.



(c) Wait until we are sure that all readers have left B and C. These nodes cannot be accessed any more.



(i) Now we can safely remove B and D

Read-Copy-Update API

- When can we safely delete old nodes? When there are no more readers
- We add an API called a *read-side critical section* that any readers of the structure must call when they read something from it
- Note that this doesn't involve mutual exclusion – it's just a way to track who's reading
- Now we know we can free a node when all the readers have left their critical sections